Defensive Programming

CMPT 145

Learning Objectives

After studying this chapter, a student should be able to:

- Describe the reasons to adopt a defensive programming attitude.
- Describe the kinds of things that might go wrong in a program.
- Apply practices for defensive programming, such as checking pre-conditions, post-conditions, and return values.
- Describe common kinds of errors encountered by first-year students working in Python.
- Describe different techniques for avoiding errors encountered by first-year students working in Python.

Defensive Programming

- Make sure that a program protects itself against
  - Incorrect or illegal data
  - Events that cause data loss
- Assume Murphy's Law is true: Whatever can go wrong, will go wrong.
  - Add code in your functions to check those things that you think can't happen.

How defensive programming helps students

- Adopting defensive practices can save time debugging.
- Defensive programming means to put code in your programs that will identify a fault as early as possible.
- Assume the worst:
  - The programmer who called your function did not read your documentation.
  - The programmer who wrote the function you are using did not honour the post-conditions.

How defensive programming helps professionals

- Software produced is more robust.
- Assume the worst:
  - Application crashes. Power outages. System crashes. User moves a file while you are sending data to it. All memory used. No more room on the disk. Internet delay. User error.

How to apply defensive programming

- Start with the assumption that everything is perfect.
  - Implement each function according to specification.
- After completing each one, add code to:
  - Check input arguments.
  - Check return values from functions you are using.
  - Look for places where something unpredictable could happen.
- A short time spent here can save a lot of time debugging.
  - You can trust Python functions if you give them the right inputs.
  - Don't trust your own functions. Check return values, and post-conditions.

How NOT to apply defensive programming

- Start by planning for every possible way things could go wrong.

What to do in CMPT 145

- Start thinking defensively.
- Use `assert` liberally.
- Always use an `else:` clause especially if your algorithm doesn't need it.
- Be aware of common errors.

What we won't try to do in CMPT 145

- You don't have to manage every kind of error in CMPT 145.
- If your function receives a value of the wrong type, let the `TypeError` halt the program.
- If the datafile is missing, or if the datafile has bad data in it, let your program crash.
- You will learn how to manage these and similar situations better in CMPT 270.

Pre- and Post-Conditions, and Return

- We've already used these concepts in our interface documentation.
- At the start of every function, check that the pre-conditions are true.
- This is part of the function, not testing.
- When you use a function, check its return value and post-conditions.

Checking pre-conditions

```
1  def factorial(n):
2      if n == 0 or n == 1:
3          return 1
4       else:
5          return n * factorial(n-1)
```

- What will this function do when given a negative integer?
- How would you handle it?

Checking pre-conditions

```
1  def factorial(n):
2      if n <= 1:
3          return 1
4       else:
5          return n * factorial(n-1)
```

- This avoids infinite recursion
- But it gives the wrong answer for negative integers
- There is no right answer for $n < 0$.

Checking pre-conditions

```
1  def factorial(n):
2      if n == 0 or n == 1:
3          return 1
4       else:
5          return n * factorial(n-1)
```

- You shouldn't assume the person calling your function is perfect.
- You shouldn't let your program enter an infinite loop.

Checking pre-conditions

- Python has a tool for this: `assert`

```
1  def factorial(n):
2      assert n >= 0, 'invalid input to factorial'
3      if n <= 1:
4          return 1
5       else:
6          return n * factorial(n-1)
```

- Syntax:

  `assert` *condition, optional-message*

- Causes Python to halt program execution with a run-time error
- Assertions can be turned off
- Use these for wolf-fencing too!

Checking pre-conditions

- Python has more general tool for this: `raise`

```
1  def fib(n):
2      assert n >= 0, 'invalid input to fib'
3      if n == 0 or n == 1:
4          return n
5      elif n > 20:
6          raise Exception('n = '+str(n)+' too big')
7      else:
8          return fib(n-1) + fib(n-2)
```

- Syntax: `raise Exception(message)`
- Causes Python to halt program execution with a run-time error
- Exceptions cannot be turned off
- More about Exceptions in CMPT 270.

Check your return value

Maybe there is something sensible you can do:

```
1  import Statistics as S
2
3  stat = S.Statistics()
4  ...
5  lowest = stat.minimum()
6
7  if lowest is not None:
8       # do the appropriate calculations
9  else:
10      # do something sensible if no data was seen
```

Check your return value

Maybe the situation is so problematic, you have to halt the whole thing:

```
1 import Statistics as S
2
3 stat = S.Statistics()
4 ...
5 lowest = stat.minimum()
6
7 assert lowest is not None, 'Statistics object added no data!'
```

Complete your conditionals

```
1  if grade > 80:
2    return 'A'
3  elif grade > 70:
4    return 'B'
5  elif grade > 60:
6    return 'C'
7  elif grade > 50:
8    return 'D'
9  elif grade >= 0:
10   return 'F'
11 else:
12   return 'this should never happen'
```

Complete your conditionals

```
1  assert grade >= 0, 'Negative grade in lecture example'
2  if grade > 80:
3    return 'A'
4  elif grade > 70:
5    return 'B'
6  elif grade > 60:
7    return 'C'
8  elif grade > 50:
9    return 'D'
10 elif grade >= 0:
11   return 'F'
```

Don't shadow Python functions

- Shadow: When you use a variable name that's defined elsewhere in scope.
- Example: `sum` is a Python function, but we can still do:

```
1 sum = 0
2 for val in my_list:
3     sum += val
```

- Python names for functions are not special.
- A name in Python has a value; sometimes the value is a function.

Know run-time exceptions

- You're not a beginner anymore.
- When Python halts your program, it tells you why.
- Know the names and potential causes for all run-time errors:

```
1 Traceback (most recent call last):
2   File "<stdin>", line 2, in <module>
3 IndexError: list assignment index out of range
```

- This is not a burden. It is valuable information.
- Google for Python exceptions. Visit Stack-Overflow.

Know scoping

- What variables are in scope?
- What variables are shadowing others?
- Is this local or global?

```
1  x = 100
2
3  def trivial_function():
4      print(x)
5      x = x + 1
6
7  trivial_function()
```

Know scoping

- What variables are in scope?
- What variables are shadowing others?
- Is this local or global?

```
1  x = 100
2
3  def trivial_function(x):
4      print(x)
5      x = x + 1
6
7  trivial_function(x)
```

List iteration

- Don't change the structure of your list while iterating
- Changing the contents is fine:

```
1 a = list(range(10))
2 i = 0
3 while i < len(a):
4     a[i] = a[i] + 1
5     i += 1
```

- Changing the structure is not good:

```
1 a = list(range(10))
2 i = 0
3 while i < len(a):
4     del a[i]
5     i += 1
```

Watch out for equality of floating point

- Floating point calculations have tiny errors due to finite precision.
- Every calculation adds a little more error
- Some calculations add a lot more error
- Two values are hardly ever equal.
- Poor:

```
1 x = 0.3
2 y = 0.1 + 0.1 + 0.1
3 if x == y:
4     print('Equal')
5 else:
6     print('Not equal!')
```

Watch out for equality of floating point

- Floating point calculations have tiny errors due to finite precision.
- Every calculation adds a little more error
- Some calculations add a lot more error
- Two values are hardly ever equal.
- Not a bad for large values:

```
1 x = 0.3
2 y = 0.1 + 0.1 + 0.1
3 if abs(x - y) < 0.000001:
4     print('Equal enough')
5 else:
6     print('Not equal enough!')
```

Watch out for equality of floating point

- Floating point calculations have tiny errors due to finite precision.
- Every calculation adds a little more error
- Some calculations add a lot more error
- Two values are hardly ever equal.
- Better for very small numbers, relative to 0.000001:

```
1 x = 0.3
2 y = 0.1 + 0.1 + 0.1
3 if abs(x - y)/max(abs(x),abs(y)) < 0.000001:
4     print('Equal enough')
5 else:
6     print('Not equal enough!')
```

Watch out for division

- Difference between integer and floating point division
- Division by zero

```
1 x = 0.3
2 y = 0.1 + 0.1 + 0.1
3 if abs(x - y)/max(abs(x),abs(y)) < 0.000001:
4     print('Equal enough')
5 else:
6     print('Not equal enough!')
```

Watch out for division by zero

- Use an if-statement if you are setting the denominator yourself.

```
1  x = 0.3
2  y = 0.1 + 0.1 + 0.1
3
4  if x == 0 and abs(y) < 0.000001:
5      print('Equal enough')
6  elif y == 0 and abs(x) < 0.000001:
7      print('Equal enough')
8  elif abs(x - y)/max(abs(x),abs(y)) < 0.000001:
9      print('Equal enough')
10 else:
11     print('Not equal enough!')
```

Watch out for division by zero

- Use an assertion if the denominator value comes from some other part of the program.

```
1  def ratio(x, y):
2      """Compute x/y for some reason.
3      Preconditions: y != 0
4      """
5      assert abs(y) > 0, 'denominator zero in ratio'
6      return x/y
```

Watch out for division by zero

- If you know why `ratio` is needed, you could avoid `assert`.

```
1  def ratio(x, y):
2      """Compute x/y for some reason.
3      If y == 0, ratio returns 0 for some reason.
4      Preconditions: x, y are numbers
5      """
6      if y == 0: return 0
7      else: return x/y
```

- This has to be appropriate for your application!

What to do in CMPT 145

- Start thinking defensively.
- Use `assert` liberally.
- Always use an `else:` clause especially if your algorithm doesn't need it.
- Be aware of common errors.